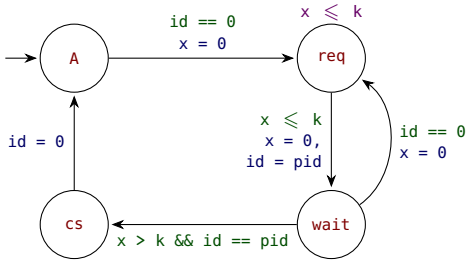
Learning Model Checking with Uppaal: A Newcomer's View of Fischer's Protocol

Version 2, 9 October 2026

Setting. A user with no background in model checking or automata opened Fischer's mutual exclusion protocol in Uppaal and asked an LLM assistant to explain it. They then learned by doing: verifying properties, breaking the model on purpose, reading counterexamples and stepping through them in the simulator. This summary describes the session from the learner's side: which concepts became clear, which were surprising, and what helped.



Template `P`, instantiated six times by **`system P;`**. Colours as in the Uppaal editor.

	What the learner did	What became clear
1	Asked for an explanation	Locations, edges, guards, updates, clocks, invariants; one template gives six processes
2	Verified the three queries	"Satisfied" is a proof over all timings, not a test run; no trace is produced when a property holds
3	Changed `x > k` to `x ⩾ k`	How the protocol depends on timing; a 6-step trace showing two processes in `cs`
4	Removed the invariant on `req`	Guards *allow*, invariants *force*; liveness and deadlock can fail while safety holds
5	Asked whether `req` always leads to `cs`	Liveness counterexamples are infinite runs; this one showed laziness, not bad luck
6	Made processes prompt	Real starvation: Fischer's protocol is safe but not fair
7	Stepped through a trace in the simulator	Symbolic clock zones; why random simulation misses the bug

Important and interesting points for a newcomer.

1. **The protocol's correctness rests on two numbers.** Every process must write its ID within `k` (`x ⩽ k`) and then wait *more* than `k` (`x > k`) before checking. Once this "last writer wins" argument was understood, the rest of the model made sense. Without the explanation, the automaton was just four circles and some guards.
2. **Breaking the model taught more than verifying it.** Seeing "satisfied" three times showed little. Changing a single character (`>` to `⩾`) and getting a concrete run with two processes in `cs` made the timing argument tangible. The race happens at one exact instant ($x = k$), which explains why exhaustive checking finds bugs that testing and random simulation miss.
3. **Guard versus invariant was the key conceptual step.** Removing the invariant on `req` was expected to break mutual exclusion; the assistant predicted this too. It didn't. The edge guard still blocked late writes. Instead, processes could linger in `req` until their only exit closed, which broke progress and caused a deadlock. Seeing an expected failure not happen was more instructive than a confirmed prediction.
4. **Safety and liveness are different questions.** *Safety* asks that nothing bad ever happens (two processes in `cs`). *Liveness* asks that something good eventually happens (a request reaches `wait`). The session showed each failing independently. A system that does nothing is perfectly safe, so both kinds must be checked.
5. **Counterexamples need careful reading.** A trace for a leads-to property `p --> q` that simply *stops* means "the system can stay here forever". The first trace for `P(1).req --> P(1).cs` was surprising: `P(1)` had *won*, but nothing forced it to enter `cs`. Only after making every process act promptly did the checker show genuine starvation: a repeating 15-step loop where others always win. Lesson: model checkers report *a* reason a property fails, often the simplest one, and modelling assumptions such as "processes may idle" matter.
6. **Symbolic traces look odd at first.** Clock values appear as ranges and relations (e.g. `P(1).x` $\in [0, 2]$, `P(2).x ⩽ P(1).x`), not single numbers. Each trace stands for infinitely many concrete timings, which is how the tool can cover continuous time in about a second.
7. **Practical Uppaal details a beginner would not guess.** Trace generation ("Diagnostic Trace") is off by default. The **`system`** line instantiates one process for each parameter value. An `urgent` channel is the standard way to force an edge to be taken as soon as it is enabled.
8. **An assistant lowers the entry barrier, but the tool has the final word.** Very short prompts ("1", "2", "experiment 3") worked because the assistant offered a numbered menu of experiments. The assistant also restored the model after each experiment, so the learner could explore without fear of losing the original. The assistant was wrong once, and the model checker caught it.

Takeaway. For a newcomer, the most effective path was *explain* → *verify* → *break* → *read the counterexample* → *refine*. Concepts that are abstract in textbooks, such as invariants, liveness, fairness and symbolic states, became concrete when each one appeared as a specific, explainable counterexample in a familiar model.